P.Vishwa Nandan(192473015)

TASK 4 – EVENT-HANDLING FORENSICS (7 Marks)

1. Aim

To analyse and correct event-handling errors in a Java AWT drawing pad application. The program must draw a dot whenever the user clicks the window, update the click count, and preserve all dots when the window is minimised, restored, or repainted.

2. Problem Description

A drawing pad application is developed using Java AWT. It is expected to perform two operations:

1. Draw a small dot wherever the user clicks.
2. Display the total number of clicks in a label.

However, users report that nothing is drawn and the click count always remains zero.

The original code is:

```
public class DrawPad extends Frame {

   int count = 0;

   Label status = new Label("Clicks: 0");

   public DrawPad() {

      add(status, BorderLayout.SOUTH);

      addMouseListener(new MouseAdapter() {

         public void mouseclicked(MouseEvent e) {

            count++;

            Graphics g = getGraphics();
```

```
            g.fillOval(e.getX(), e.getY(), 6, 6);
        }
    });

    setSize(400, 300);

    setVisible(true);
  }
}
```

The program contains errors in event handling, label updating, and drawing persistence.

3. Task 4.1 – Locate the Defect (2 Marks)

3.1 Defect 1 – Incorrect Method Name

The original program uses:

public void mouseclicked(MouseEvent e)

But Java's MouseAdapter class provides the method:

public void mouseClicked(MouseEvent e)

The difference is the capital letter C in mouseClicked.

Java is case-sensitive. Therefore, mouseclicked() and mouseClicked() are treated as different method names.

The incorrectly named method does not override the original MouseAdapter method.

Problem: When the user clicks the window, Java calls mouseClicked(), not mouseclicked(). Therefore, the click-handling code is never executed.

Solution:

```
@Override
public void mouseClicked(MouseEvent e) {
  // Event handling code
}
```

3.2 Defect 2 – Label Is Never Updated

The original program contains:

count++;

This increases the click counter, but it does not update the label.

The label is initially created with:

Label status = new Label("Clicks: 0");

There is no statement to change its displayed text.

Problem: Even after correcting the mouse event method, the label continues displaying Clicks: 0.

Solution:

status.setText("Clicks: " + count);

This updates the label whenever a click occurs.

3.3 Defect 3 – Direct Use of getGraphics()

The original program uses:

Graphics g = getGraphics();

g.fillOval(e.getX(), e.getY(), 6, 6);

The getGraphics() method provides a temporary drawing context for the component.

Drawing directly through this graphics object does not preserve the dot as part of the component's painting state.

Problem: Dots may disappear when the window is repainted.

Solution: Store every clicked point in an ArrayList and redraw all stored points inside the paint(Graphics g) method.

3.4 Summary of Defects

No.	Defect	Problem
1	Incorrect mouseclicked() method name	Click event is not handled
2	Missing status.setText()	Click count is not displayed
3	Direct use of getGraphics()	Drawn dots are not persistent
4	No stored drawing information	Previous dots cannot be restored during repainting

4. Task 4.2 – Construct a Failing Interaction (2 Marks)

4.1 Steps to Reproduce the Problem

Even after correcting the mouse event method name, the dots may disappear when the window is minimised and restored.

The following steps demonstrate the problem:

1. Open the drawing pad application.
2. Click five different locations in the drawing area.
3. Observe that five dots are displayed.
4. Minimise the application window.
5. Restore the application window.
6. Observe that the previously drawn dots have disappeared if they were drawn directly using getGraphics().

4.2 Why Does This Happen?

When the window is minimised, restored, resized, or covered by another window, AWT may request the component to repaint itself.

During repainting, Java calls the component's paint(Graphics g) method.

However, if the application has drawn dots only using getGraphics(), it has not stored the dot positions anywhere.

Therefore, when repainting occurs, Java redraws the normal component contents but does not know where the previous dots were located.

As a result, the dots disappear.

4.3 Correct Solution

The application must maintain the drawing information separately from the screen.

The following steps solve the problem:

1. Store each mouse-click position as a Point object.
2. Add the point to an ArrayList.
3. Update the click-count label.
4. Call repaint().
5. Override paint(Graphics g).
6. Draw every stored point again whenever repainting occurs.

This ensures that the dots remain visible even after the window is minimised, restored, or resized.

5. Task 4.3 – Repair the Program (2 Marks)

5.1 Corrected Program Design

The corrected program uses the following components:

Frame: Creates the main drawing pad window.

Label: Displays the total number of clicks.

MouseAdapter: Handles mouse-click events.

ArrayList<Point>: Stores the position of every mouse click.

Graphics: Draws the dots on the drawing area.

paint(Graphics g): Redraws all stored dots whenever the component needs repainting.

repaint(): Requests AWT to repaint the component.

5.3 Explanation of TODO 1 – mouseClicked()

```
@Override
public void mouseClicked(MouseEvent e) {

   points.add(e.getPoint());

   count++;

   status.setText("Clicks: " + count);

   repaint();
}
```

Explanation:

1. mouseClicked() is automatically called when the user completes a mouse click.
2. e.getPoint() retrieves the mouse-click position.
3. points.add() stores the clicked position in the ArrayList.
4. count++ increases the total click count.
5. status.setText() updates the label.
6. repaint() requests the window to redraw its contents.

5.4 Explanation of TODO 2 – paint(Graphics g)

```
@Override
public void paint(Graphics g) {

   super.paint(g);

   for (Point p : points) {

      g.fillOval(
         p.x - 3,
         p.y - 3,
         6,
         6
      );
   }
}
```

Explanation:

1. paint(Graphics g) is called whenever AWT needs to draw the component.
2. super.paint(g) allows the normal component and its child components to be painted.
3. The for-each loop visits every stored point.
4. p.x represents the horizontal position.
5. p.y represents the vertical position.
6. p.x - 3 and p.y - 3 position the upper-left corner of the circle so that its centre matches the clicked point.
7. fillOval() draws a filled circle of size 6 × 6 pixels.

Because the clicked positions are stored in the ArrayList, all dots can be drawn again whenever repainting occurs.

5.5 Expected Output

Initial Window:

Draw Pad

Drawing Area

Clicks: 0

After Three Mouse Clicks:

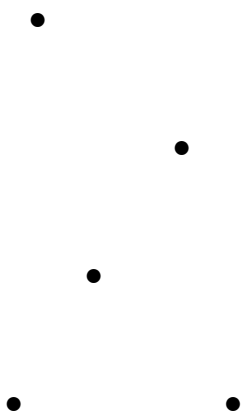
Draw Pad



Clicks: 3

After Five Mouse Clicks:

Draw Pad



Clicks: 5

After Minimising and Restoring:

The same five dots should remain visible, and the label should continue displaying:

Clicks: 5

6. Task 4.4 – Prevent Recurrence (1 Mark)

6.1 Role of the @Override Annotation

The @Override annotation tells the Java compiler that a method is intended to override a method from its parent class or implement a method from an interface.

In the original program, the method was incorrectly written as:

public void mouseclicked(MouseEvent e)

When the annotation is added:

@Override

public void mouseclicked(MouseEvent e)

The compiler produces an error similar to:

error: method does not override or implement

a method from a supertype

@Override

^

1 error

This happens because MouseAdapter does not contain a method named mouseclicked().

The correct method is:

@Override

public void mouseClicked(MouseEvent e)

6.2 Why Is @Override Important?

1. It identifies incorrect method names.
2. It prevents accidental method overloading when overriding was intended.
3. It detects signature mismatches.
4. It improves code readability.
5. It helps detect errors during compilation instead of waiting for runtime.

Conclusion: The @Override annotation would have detected the incorrect method name before the program was executed.

7. Complete the Code – TODO 1 and TODO 2

The assignment provides a partially completed DrawPad program.

TODO 1 – Correct Mouse Event Handler

Replace the TODO comment with:

@Override

public void mouseClicked(MouseEvent e) {

```
    points.add(e.getPoint());

    status.setText("Clicks: " + points.size());

    repaint();
}
```

TODO 2 – Draw Every Stored Point

Replace the TODO comment inside paint(Graphics g) with:

for (Point p : points) {

```
    g.fillOval(
        p.x - 3,
        p.y - 3,
        6,
        6
    );
}
```

The complete program is provided in Section 5.2.

7.1 Expected Verification

Action	Expected Result
Open the drawing pad	Clicks: 0
Click once	One dot appears; Clicks: 1
Click four more times	Five dots appear; Clicks: 5
Minimise the window	Window is minimised
Restore the window	All five dots remain visible
Click again	Sixth dot appears; Clicks: 6

8. Alternative Concept and Complexity Analysis

8.1 Alternative – Using BufferedImage

The original corrected solution stores the clicked positions in an ArrayList and redraws every point whenever repainting occurs.

Another approach is to use an off-screen BufferedImage.

A BufferedImage acts as an image stored in memory. Whenever the user clicks, the program draws the dot directly onto the image.

During repainting, the program copies the complete image onto the screen.

Working

1. Create a BufferedImage.
2. When the user clicks, obtain the image's Graphics2D object.

3. Draw the dot onto the image.
4. Dispose of the Graphics2D object.
5. Request repainting.
6. In paint(Graphics g), display the stored image.

This approach stores the completed drawing rather than storing individual click positions.

8.3 Comparison of Both Approaches

Approach 1 – ArrayList of Points

Each mouse click is stored as a Point object.

During repainting, the application loops through all points and draws them again.

Approach 2 – BufferedImage

Each mouse click is drawn directly onto an image stored in memory.

During repainting, the program displays the stored image rather than redrawing every individual point.

8.4 Time Complexity Analysis

Let:

- n = Number of clicks.
- W = Drawing area width.
- H = Drawing area height.

ArrayList Approach:

Adding a new point:

O(1) amortized

Repainting:

O(n)

The application must visit every stored point and draw it again.

BufferedImage Approach:

Drawing a new dot:

O(1)

A repaint uses one drawImage() operation instead of iterating through all stored points.

The work required to copy or render the image generally depends on its dimensions:

$O(W \times H)$

This work is independent of the number of clicks. Actual rendering time depends on the graphics implementation and the area that needs updating.

8.5 Space Complexity Analysis

ArrayList Approach:

Each click stores one Point object containing its coordinates.

Space complexity:

$O(n)$

Memory increases as the number of clicks increases.

BufferedImage Approach:

The image stores pixel information for the entire drawing area.

Space complexity:

$O(W \times H)$

For an ARGB image, a typical pixel requires four bytes.

Approximate image memory:

$4 \times W \times H$ bytes

The image requires memory even when only a few dots have been drawn.

8.6 Comparison Table

Feature	**ArrayList Approach**	**BufferedImage Approach**
Drawing storage	Point objects	Image pixels
Click operation	Add point	Draw onto image
Time per click	O(1) amortized	O(1) for fixed-size dot
Repaint work	O(n)	Approximately $O(W \times H)$ full-image rendering
Additional space	O(n)	$O(W \times H)$
Persistent drawing	Yes	Yes

Best suited for	Small or moderate drawings	Large drawings with frequent repainting

8.7 Engineering Decision

The ArrayList approach is suitable for the given drawing pad because it is simple, easy to understand, and stores only the clicked positions.

The BufferedImage approach is more suitable when the drawing contains many points and repainting occurs frequently.

Therefore, the choice depends on the size of the drawing and the application's memory and performance requirements.

9. Testing and Verification

The following test cases verify the corrected program.

Test Case	Action	Expected Result
TC1	Open the application	Clicks: 0
TC2	Click once	One dot appears
TC3	Click five times	Five dots appear
TC4	Click ten times	Clicks: 10
TC5	Minimise the window	Window is minimised
TC6	Restore the window	All previous dots remain
TC7	Cover and uncover the window	Previous dots remain
TC8	Resize the window	Previous dots remain visible
TC9	Close the application	Window closes successfully
TC10	Use incorrect method with @Override	Compilation error is produced

9.1 Required Screenshot Evidence

Capture the following screenshots after executing the program.

Screenshot 1 – Initial Window

The application should display:

Clicks: 0

Screenshot 2 – After Drawing

Show several dots and the updated click count.

Screenshot 3 – After Minimising and Restoring

Show that all previously drawn dots remain visible and the click count is unchanged.

Suggested folder:

outputs/

│

├── drawpad-initial.png

├── drawpad-after-clicks.png

└── drawpad-after-restore.png

10. Final Result

The event-handling defects in the original Java AWT drawing pad were identified and corrected.

The incorrect mouseclicked() method was replaced with the correct mouseClicked() method. The click count was updated using Label.setText().

The program was redesigned to store clicked positions using an ArrayList and redraw them inside paint(Graphics g). This ensures that the dots remain visible after repainting, minimising, restoring, and resizing the window.

The @Override annotation was also explained as a method of detecting incorrect method signatures at compile time.

Finally, the ArrayList approach was compared with the BufferedImage approach using time and space complexity.

Thus, the drawing pad demonstrates correct mouse-event handling and persistent drawing in Java AWT.

.